

Producer-Consumer Problem

- Paradigm for cooperating processes, *producer* process produces information that is consumed by a *consumer* process.
- A solution for this problem is implemented using a **shared buffer**, this buffer can be:
  - *unbounded-buffer* places no practical limit on the size of the shared buffer
  - *bounded-buffer* assumes that there is a fixed buffer size
    - If the buffer empty the consumer must wait.
    - If the buffer is full the producer must wait.
- We have to prevent the consumer from reading the buffer value unless the producer finished his job and produce the output for the consumer.





Interprocess Communication (IPC)

- Mechanism for processes to communicate and to synchronize their actions
- Message system (**Message passing)** – processes communicate with each other without resorting to shared variables
- IPC facility provides two operations:
  - **send**(*message*) – message size fixed or variable
  - **receive**(*message*)
- If *P* and *Q* wish to communicate, they need to:
  - establish a *communication link* between them
  - exchange messages via send/receive





Implementation Questions

- How are links established?
- Can a link be associated with more than two processes?
- How many links can there be between every pair of communicating processes?
- What is the capacity of a link?
- Is the size of a message that the link can accommodate fixed or variable?
- Is a link unidirectional or bi-directional?





Naming

- **Processes that want to communicate must have a way to refer to each other. They can use:**
  - Direct communication
  - Indirect communication





Direct Communication

- Processes must name each other explicitly:
  - **send** (*P, message*) – send a message to process P
  - **receive**(*Q, message*) – receive a message from process Q
- Properties of communication link
  - Links are established **automatically**
  - A link is associated with **exactly one pair** of communicating processes
  - Between each pair there exists **exactly one link**
  - The link may be unidirectional, but is usually bi-directional
- **Direct communication can be divided to:**
  - Symmetry in addressing: both processes (sender and receiver) should name each other. (usually needed for bidirectional communications).
  - Asymmetry in addressing: only the sender names the recipient. (usually needed for unidirectional (one direction) communications.





Indirect Communication

- Messages are directed and received from mailboxes (also referred to as ports)
  - Each mailbox has a unique id
  - Processes can communicate only if they share a mailbox
- Properties of communication link
  - Link established **only if** processes share a common mailbox
  - A link may be associated with **many processes**
  - Each pair of processes may share **several communication links**
  - Link may be unidirectional or bi-directional





Indirect Communication

- Operations
  - create a new mailbox
  - send and receive messages through mailbox
  - destroy a mailbox
- Primitives are defined as:

  send(*A, message*) – send a message to mailbox A

  receive(*A, message*) – receive a message from mailbox A





Indirect Communication

- Mailbox sharing
  - P_1, P_2, and P_3 share mailbox A
  - P_1, sends; P_2 and P_3 receive
  - **Who gets the message????**
- Solutions
  - Allow a link to be associated with at most two processes
  - Allow only one process at a time to execute a receive operation (blocking ??? See the next slides).
  - Allow the system to select arbitrarily the receiver. Sender is notified who the receiver was.





Synchronization

- Message passing may be either blocking or non-blocking
- **Blocking** is considered **synchronous**
  - **Blocking send** has the sender block until the message is received
  - **Blocking receive** has the receiver block until a message is available
- **Non-blocking** is considered **asynchronous**
  - **Non-blocking send** has the sender send the message and continue
  - **Non-blocking receive** has the receiver receive a valid message or null





Buffering

- Whether communication is direct or indirect, messages exchanged by communicating processes reside in a temporary queue, such queue can be implemented in three ways.
- Queue of messages attached to the link; implemented in one of three ways
  1. Zero capacity – 0 messages
     Sender must wait for receiver (rendezvous) (Blocking send and Blocking receive)
  2. Bounded capacity – finite length of *n* messages
     Sender must wait if link full (Blocking send), if the link is not full, then non-blocking send.
  3. Unbounded capacity – infinite length
     Sender never waits.





Client-Server Communication

- Sockets
- Remote Procedure Calls
- Remote Method Invocation (Java)





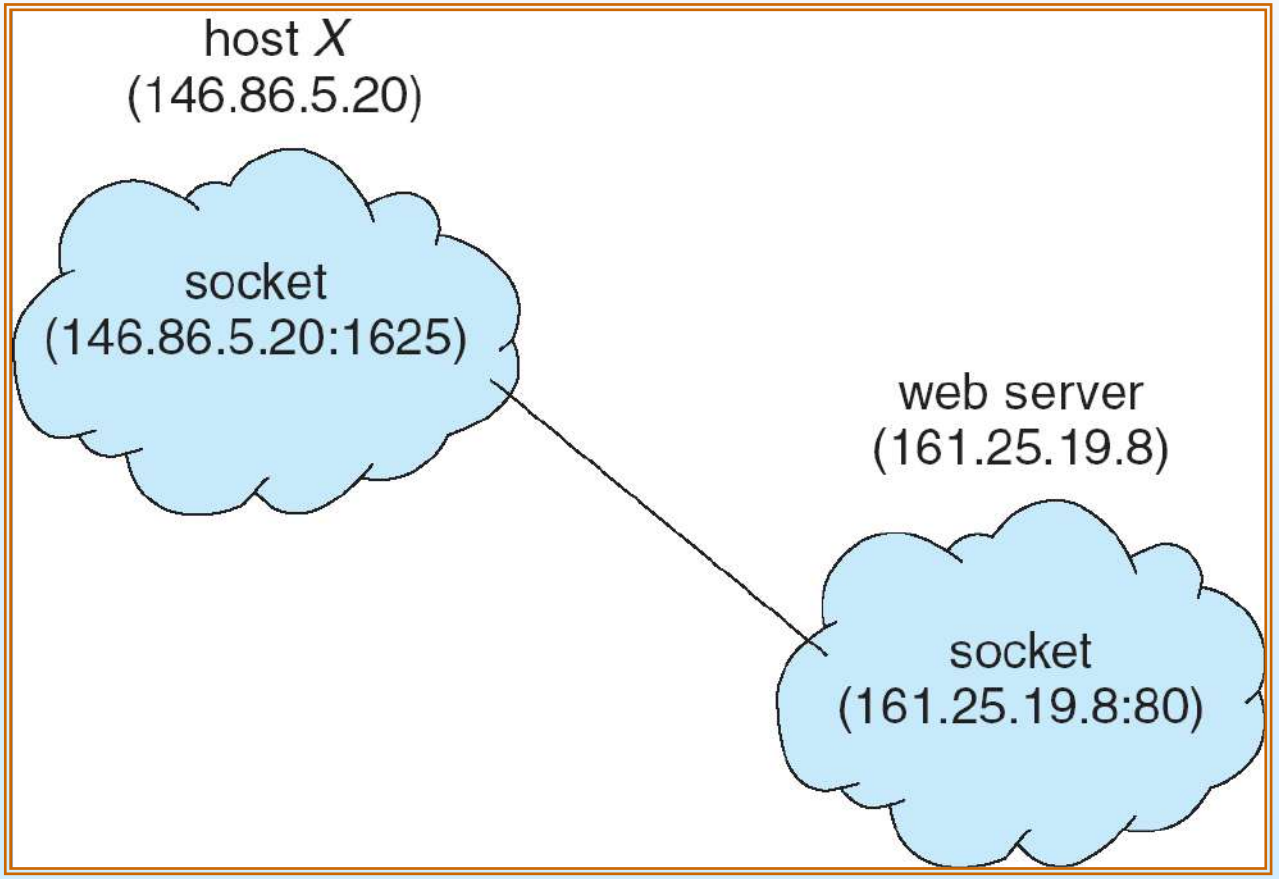
Sockets

- A socket is defined as an *endpoint for communication*
- **Concatenation of IP address and port**
- The socket **161.25.19.8:1625** refers to port **1625** on host **161.25.19.8**
- Communication consists between a pair of sockets





Socket Communication





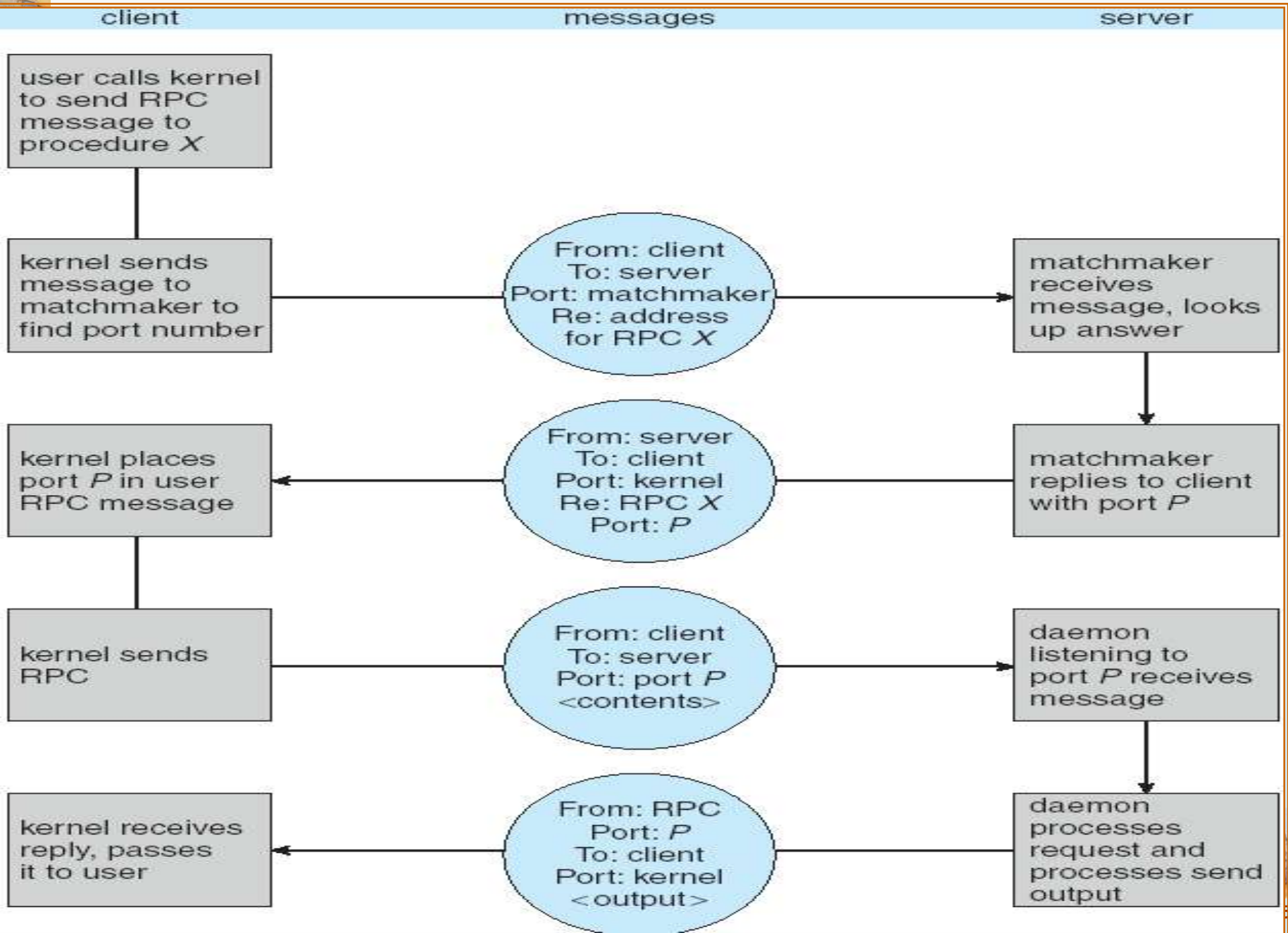


Remote Procedure Calls

- Remote procedure call (RPC) abstracts procedure calls between processes on networked systems, and it is similar to IPC but in RPC the processes are not at the same machine while they are in IPC.
- **Stubs** – client-side proxy for the actual procedure on the server.
- Stubs in RPC system used to hide the details that allow communication to take place. Usually each RPC has a separate stub.
  - Stub locates the port on the server for the remote procedure.
  - Stub marshals the parameters for the remote procedure.
- The client-side stub locates the server and *marshalls* (*packaging the parameters into a form that can be transmitted over a network*) the parameters.
- The server-side stub receives this message, unpacks the marshalled parameters, and peforms the procedure on the server.



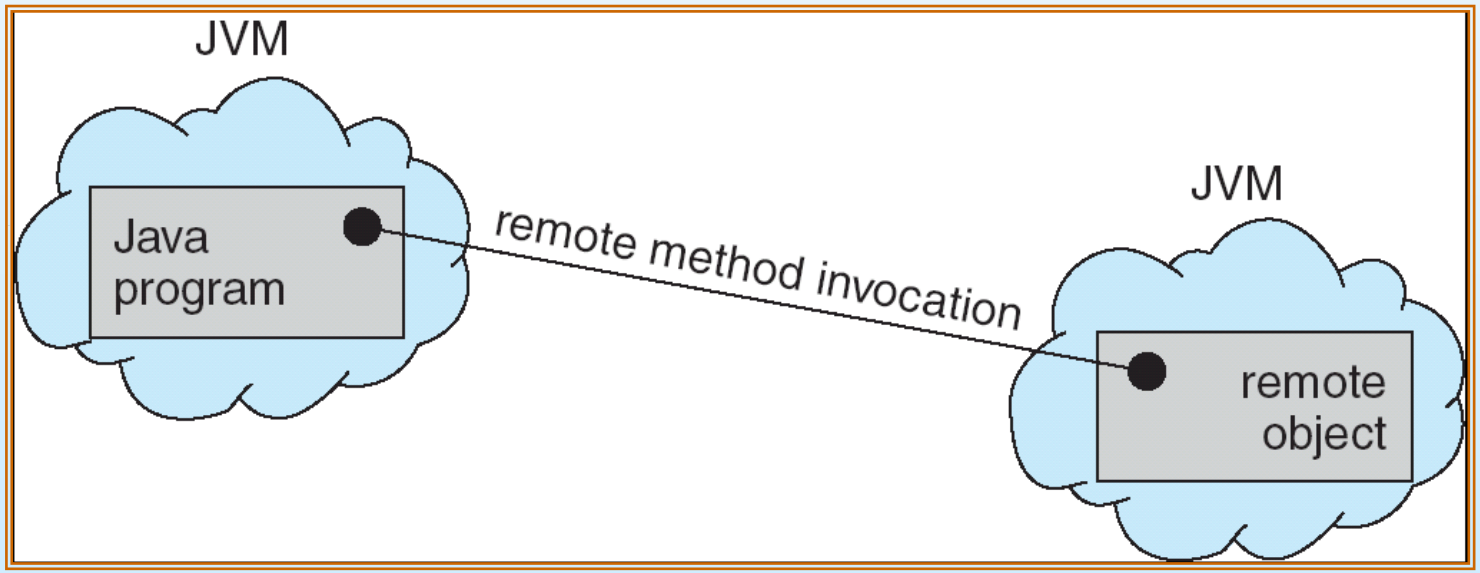
Execution of RPC





Remote Method Invocation

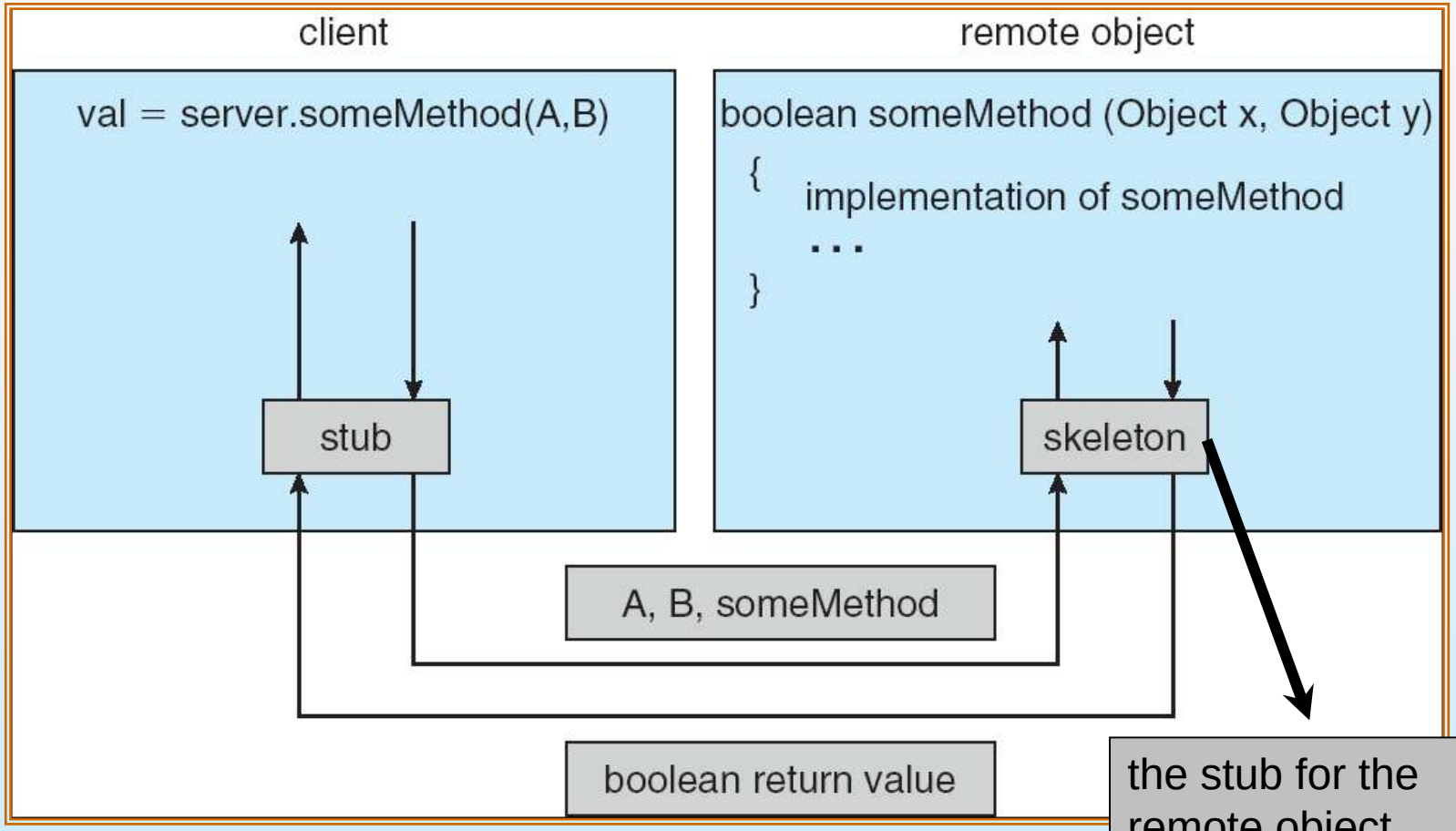
- Remote Method Invocation (RMI) is a Java mechanism similar to RPCs.
- RPCs usually implemented in C++.
- RMI allows a Java program on one machine to invoke a method on a remote object.







Marshalling Parameters



the stub for the remote object called skeleton in java terminology

End of Chapter 3

